

Sub-module 9

Waveform Analysis Using Testbenches

From RTL Design to Simulation-Ready Waveforms

What We'll Cover

01 | Testbench Fundamentals

02 | Project Testbench Files

03 | VCD vs FST Format

04 | Lab Commands

05 | Signal Grouping by Layer

06 | AXI Handshake in GTKWave

07 | UART Frame Decode

08 | AXI Handshake in GTKWave

09 | Debug Patterns

What is a Testbench?

Analogy: Think of your SoC as a new car engine, and the testbench as the diagnostic rig in the factory. It provides the fuel (clock) and presses the pedals (stimuli) to ensure the engine runs correctly before it ever hits the road.

What It Is	What It Does	Why It Matters
• An extra Verilog module that wraps your DUT (Device Under Test) • Simulation-only — never synthesized to real hardware • Sits outside the design and talks to it via ports	• Drives clock, reset, and bus stimuli into the DUT • Observes outputs via waveforms (VCD/FSDB) and logs • Uses assertions (\$assert) to auto-check correctness	• Catches bugs before expensive silicon is made • Simulates real-world conditions safely • Required step before tapeout in any SoC project

Testbench Skeleton

Step	Task	Generic Verilog	THIS Project (tb_top.v)
01	Define Module	<code>module tb_soc;</code>	<code>module tb_top; // no ports, simulation-only</code>
02	Generate Clock	<code>always #5 clk = ~clk</code>	<code>always #10 clk = ~clk // 50 MHz (10ns half-period)</code>
03	Apply Reset	<code>rstn=0 for N cycles</code>	<code>reset<=1; repeat(10) @(posedge clk); reset<=0;</code>
04	Instantiate DUT	<code>soc_top DUT (...)</code>	<code>top u_dut(.clk(clk),.reset(reset),.uart_tx(uart_tx_pin),.uart_rx(uart_rx_pin));</code>
05	Preload Memory	<code>\$readmemh("prog.hex",rom)</code>	ROM gets hex via <code>+romhex=rtl/rom.hex plusarg</code> → DUT's <code>rom.v</code> calls <code>\$readmemh</code>
06	UART Monitor	N/A in generic TB	Independent initial begin: <code>uart_monitor</code> – detects falling edge, counts bit periods, builds byte → <code>rx_fifo</code>

The Simulation Flow

make soc → VCD → FST → GTKWave → Iterate

1 COMPILE	2 RUN	3 CONVERT	4 INSPECT	5 ITERATE
<pre>make soc</pre> Verilator --binary --trace 9 RTL files + tb/tb_top.v → obj_soc/sim_soc	<pre>./obj_soc/sim_soc +romhex=rtl/rom.hex</pre> Executes simulation Output: tb_top.vcd (147 MB raw VCD)	<pre>vcd2fst tb_top.vcd tb_top.fst</pre> Compress: 147 MB → ~5 MB (28× smaller) Faster GTKWave loading	<pre>make wave OR gtkwave tb_top.fst</pre> GTKWave opens tb_top.fst 5-layer signal analysis	Fix RTL → make soc → make wave Repeat until: ✓ TEST PASSED in console

⚠ Icarus Verilog, ModelSim, and QuestaSim are NOT used in this course — use Verilator only.

Waveform VCD File Size Reference

make pico	tb_picorv32.vcd	36 KB
make axi_sa	tb_axi_lite_standalone.vcd	3.8 KB
make uart	tb_uart.vcd	2.9 MB
make riscv_axi	tb_riscv_axi_lite.vcd	3.3 KB
make axi_uart	tb_axi_lite.vcd	1.9 MB
make soc	tb_top.vcd → tb_top.fst	147 MB → ~5 MB

GTKWave Setup: Opening THIS Project's Waveforms

Step 1 — Run Simulation

```
make soc          # compiles + runs + auto-opens
GTKWave
# OR manually:
vcd2fst tb_top.vcd tb_top.fst
gtkwave tb_top.fst &
Both commands produce tb_top.fst (compressed for speed)
```

Step 2 — Load Signal Hierarchy (SST Panel)

```
Expand: tb_top → u_dut (top module)
├── (CPU signals: b_state, m_axi_*)
├── u_cpu (picorv32 core)
├── u_interconnect (wr_sel, rd_sel)
├── u_uart (buf_valid, aw_got, w_got)
│   ├── u_tx (state, clk_cnt, tx)
│   └── u_rx (state, data_valid)
└── u_rom, u_sram
```

Step 3 — GTKWave Navigation Shortcuts

Ctrl+A	Zoom to fit all time
Ctrl+Scroll	Zoom in/out at cursor
T	Place time marker
Ctrl+G	Go to specific time (ns)
Right-click → Highlight	Color-code signal groups

Waveform Files by Lab

```
make pico      → tb_picorv32.vcd      (36 KB)
make axi_sa    → tb_axi_lite_standalone.vcd (3.8 KB)
make uart      → tb_uart.vcd          (2.9 MB)
make riscv_axi → tb_riscv_axi_lite.vcd  (3.3 KB)
make axi_uart  → tb_axi_lite.vcd       (1.9 MB)
make soc       → tb_top.fst            (~5 MB)
```

💡 Save your layout: File → Write Save File → waveform.gtkw | Reload: gtwave tb_top.fst -a waveform.gtkw

Once signals are grouped by layer, the layout persists across simulation runs — you only have to arrange them once.

Layer-by-Layer GTKWave Analysis

L1 System Health	L2 CPU Activity	L3 AXI Bridge FSM	L4 Interconnect Routing	L5 Peripheral Detail (UART)
- clk – clock toggling? - reset – released after ~10 cycles? - uart_tx_pin – transitions visible?	- u_cpu.mem_valid – asserted on every access - u_cpu.mem_instr – HIGH=fetch, LOW=load/store - u_cpu.mem_addr – PC during fetch; data addr during LW/SW	- b_state[2:0] – 0=IDLE, 1=WR_AW, 3=RD_AR, 4=RD_R - m_axi_awvalid – write address presented - m_axi_bvalid – write response from slave - m_axi_rvalid – read data from slave	- wr_sel[2:0] – one-hot: 001=ROM 010=SRAM 100=UART - rd_sel[2:0] – same for reads - wr_active – write transaction in progress - rd_active – read transaction in progress	- uart_axi.buf_valid – TX byte buffered - uart_axi.u_tx.state – 0=IDLE 1=START 2=DATA 3=STOP - uart_axi.u_tx.clk_cnt – baud counter (0-433) - uart_axi.rx_buf_valid – RX byte waiting to be read

Pattern: How to find a UART Byte Transmission

1 Find `wr_sel ≠ 0 AND m_axi_awaddr = 0x10000000`

This identifies a UART TX write transaction

2 Follow `buf_valid = 1 → u_tx.state: 0→1→2→3→...→0`

TX state machine cycles through IDLE→START→DATA→STOP

3 Zoom into `uart_tx_pin: 10 bit-periods = one complete frame`

1 start bit + 8 data bits + 1 stop bit = 4340 cycles = 86.8 μs

Hands-On: Decode 'H' from uart_tx_pin

1 Find Start Bit

Locate the falling edge of uart_tx_pin (goes 1→0)
Press T to place Marker 1 here
This is the beginning of the UART frame

2 Sample Each Data Bit at Mid-Period

Each bit = 434 cycles = 8.68 μ s
Sample at Marker1 + 4.34 μ s (half-bit offset) for D0, then +8.68 μ s for each subsequent bit

D0: +4.34 μ s → tx=0	D4: +39.06 μ s → tx=0
D1: +13.02 μ s → tx=0	D5: +47.74 μ s → tx=0
D2: +21.70 μ s → tx=0	D6: +56.42 μ s → tx=1
D3: +30.38 μ s → tx=1	D7: +65.10 μ s → tx=0

3 Build the Byte

Collect D7:D0 in order (LSB first, but displayed D7→D0):

D7 D6 D5 D4 D3 D2 D1 D0 → 0 1 0 0 1 0 0 0
= 0x48 = 72 decimal = 'H' ✓

4 Verify Stop Bit

After D7, uart_tx_pin returns to 1 for exactly 8.68 μ s (stop bit), then stays HIGH (idle line). If the line doesn't return high, a framing error has occurred.



This is exactly what the UART monitor in tb_top.v does automatically — the uart_monitor initial block decodes each byte the same way!

Timing Reference Table

Event	Time	Cycles
Reset released	~0 ns	0
First instruction fetch	~200 ns	10
First UART TX 'H' starts	~3–5 μ s	150–250
UART start bit duration	8.68 μ s	434
UART full frame (8N1, 10 bits)	86.8 μ s	4,340
UART byte with overhead	~90 μ s	~4,500
"Hello Deepak from Nielit!" (26 chars)	~2.3 ms	~115,000
10 greeting messages	~23 ms	~1,150,000
PING phase start	~23 ms	~1,150,000
4 PING bytes (P,I,N,G)	~0.4 ms	~20,000
TEST PASSED logged	~23.4 ms	~1,170,000
\$finish (Verilator)	~23.4 ms	—
Wall time (Verilator execution)	~1.2 s	—

Formula: $\text{cycles_per_bit} = \text{CLK_FREQ} / \text{BAUD_RATE} = 50,000,000 / 115,200 = 434$ | $\text{frame_cycles} = 434 \times 10 \text{ bits} = 4,340$ | $\text{frame_time} = 4,340 \times 20\text{ns} = 86.8 \mu\text{s}$

Reading AXI Waveforms



Golden Rule: A transfer occurs **ONLY** when both **VALID=1 AND READY=1** on the same rising clock edge — neither alone is sufficient.

AW Write Address	W Write Data	B Write Response	AR Read Address	R Read Data
Master: m_axi_awvalid / m_axi_awready Slave: s2_axi_awvalid / s2_axi_awready CPU → Interconnect → UART Slave Presents write address 0x10000000	Master: m_axi_wvalid / m_axi_wready Slave: s2_axi_wvalid / s2_axi_wready Carries actual byte value wstrb=4'h1 for byte lane 0	Master: m_axi_bready Slave: s2_axi_bvalid Slave acknowledges write done b_state returns to IDLE	Master: m_axi_arvalid / m_axi_arready Slave: s2_axi_arvalid / s2_axi_arready Read from STATUS: 0x10000008 Read from RX data: 0x10000004	Master: m_axi_rvalid / m_axi_rready Slave: s2_axi_rvalid / s2_axi_rready Returns register value to CPU b_state → RD_R → IDLE

Routing Signals & Address Map

Signal	Value	Meaning
wr_sel[2:0]	3'b001	ROM selected for write
wr_sel[2:0]	3'b010	SRAM selected for write
wr_sel[2:0]	3'b100	UART selected for write
m_axi_awaddr	0x10000000	UART TX register (write target)
m_axi_araddr	0x10000008	UART STATUS register (poll RX ready)
m_axi_araddr	0x10000004	UART RX data register

CPU Fetch vs. UART Transmission

CPU Instruction Fetch

```
cpu_mem_valid = 1
cpu_mem_instr = 1          // fetch active
cpu_mem_addr[31:0]        // incrementing by 4
    e.g. 0x0 → 0x4 → 0x8 → 0xC ...

b_state[2:0]: 0 → 3 → 4 → 0
    IDLE → RD_AR → RD_R → IDLE

rd_sel[2:0] = 3'b001      // ROM selected
cpu_mem_rdata: RISC-V opcode
    e.g. 0x00020137 = LUI x2, 0x20
```

Key Pattern:

Every ROM read: b_state cycles 0→3→4→0 in 2 clock cycles. rd_sel = 3'b001 confirms ROM is the target. Watch cpu_mem_addr increment by 4 (word-aligned RISC-V instructions).

Reset → First fetch: ~200 ns (10 cycles)
Boot complete: ~3–5 μs (150–250 cycles)

UART Byte Transmission

```
cpu_mem_addr = 0x10000000
wstrb = 4'h1          // byte lane 0

b_state: 0 → 1 → 0      // IDLE→WR_AW→IDLE
wr_sel = 3'b100        // UART selected

uart_axi.buf_valid = 1  // TX buffer loaded
uart_axi.u_tx.state:
    0→1→2→3→0 (IDLE→START→DATA→STOP→IDLE)

uart_tx_pin:
    falling edge (start) → 8 bits → high (stop)
```

Key Pattern:

UART TX write: b_state cycles 0→1→0 (simpler than read — no read-data phase). wr_sel = 3'b100 confirms UART. buf_valid pulses high when TX byte is latched.

1 bit = 434 cycles = 8.68 μs
1 frame = 4340 cycles = 86.8 μs (8N1)
10 msg = ~23 ms = ~1,150,000 cycles

Interactive Debugging

BUG 1

Simulation passes but no '[TB] RX byte:' output

ROOT CAUSE

byte_logger block not synced — delta-cycle race on rx_fifo

FIX / DEBUG STEPS

```
independent byte_logger initial
block in tb_top.v
while (log_ptr == rx_wr_ptr)
#10; // wait for new fifo
$display("[TB] RX byte: %c",
rx_fifo[log_ptr & 63]);
```

LESSON LEARNED

Concurrent Verilog blocks can have delta-cycle races;
#10 delay separates logger from monitor safely.

BUG 2

Bridge stuck in state ST_WR_AW (b_state = 1 forever)

ROOT CAUSE

Interconnect wr_sel = 3'b000 — address not decoded

FIX / DEBUG STEPS

In GTKWave: check wr_sel[2:0] when awaddr is presented
Verify axi_decoder.v casez pattern matches the address
Expected: wr_sel = 3'b100 for addr 0x10000000

LESSON LEARNED

A hung b_state means the slave's awready never arrived — always check wr_sel routing first.

BUG 3

TEST FAILED — echo byte wrong or zero

ROOT CAUSE

uart_rx_pin driven wrong — testbench timing off

FIX / DEBUG STEPS

Check uart_axi.u_rx.data_valid pulse width in waveform
Check uart_axi.rx_buf_data value at data_valid=1
Verify TB sends PING with correct BAUD_FULL interval

LESSON LEARNED

TX→RX loopback timing must account for uart_rx latency (4340 cycles = 86.8 μs per byte).

Interview Q: "You see b_state stuck at 1 — what do you check?" → **Answer:** Check wr_sel routing → slave awready response → slave FSM → backpressure condition

Interview Q: "How do you verify an AXI write completed?" → **Answer:** AWVALID+AWREADY=1 (AW done), WVALID+WREADY=1 (W done), BVALID+BREADY=1 (B done)

Sub-module 9 — Complete Lab Reference

Sub-module	Command	VCD / FST File	Key Signals
M1	make pico	tb_picorv32.vcd (36 KB)	mem_valid, mem_instr, mem_addr
M2	make axi_sa	tb_axi_lite_standalone.vcd (3.8 KB)	awvalid, awready, rdata
M3	make uart	tb_uart.vcd (2.9 MB)	tx, rx, state, clk_cnt
M4	make soc_pico_uart_mem	tb_mem_soc.fst (142 MB raw)	cpu_mem_addr, mem_wstrb
M5	make riscv_axi	tb_riscv_axi_lite.vcd (3.3 KB)	b_state, m_awvalid, s_rdata
M6	make axi_uart	tb_axi_lite.vcd (1.9 MB)	buf_valid, aw_got, u_tx.state
M7	make fw + wave	tb_top.fst	rom fetch trace in waveform
M8	make soc	tb_top.fst (~5 MB)	full 5-layer analysis
M9 ★	make wave	tb_top.fst (~5 MB)	all layers + UART decode exercise

Interview Questions — Waveform Analysis

Q1: How do you verify an AXI write completed?

A: AWVALID+AWREADY=1 → WVALID+WREADY=1 → BVALID+BREADY=1

Q2: How do you confirm a UART byte was transmitted correctly?

A: Count falling edge, sample 8 bits at CLKS_HALF + n×CLKS_PER_BIT, compare to expected value

Q3: You see b_state stuck at 1 — what do you check?

A: wr_sel routing → slave awready → slave FSM state → backpressure condition